DEVELOPMENT PACKAGE 2

Authentication & Identity

Project: Bidra Platform
Package: 2
Version: 1.0
Purpose: Build the security and identity foundation

---

1. Objective

Build the complete Authentication & Identity module for Bidra.

This package creates the identity layer that every future platform feature depends on.

It must support secure user registration, login, logout, session management, roles, permissions and audit logging.

This package must build on Development Package 1.

---

2. AI Role

Act as the Lead Principal Software Engineer for Bidra.

Your job is to implement a secure, production-ready identity foundation.

Do not implement Organizations, Campaigns, Needs, Contributions, Payments, Volunteers, Time Credits, Rewards or Admin dashboards in this package.

---

3. Specifications to Use

Before coding, load and follow:

- Volume 10: Engineering Standards
- Volume 11: AI Development Instructions
- Volume 12 Part 1: AI Master System Prompt
- Volume 12 Part 5: Authentication & Identity Module Build Prompt
- Volume 13: Development Roadmap & Sprint Plan
- Development Package 1 output

---

4. Scope

This package includes:

- User accounts
- Registration
- Login
- Logout
- Email verification
- Password reset
- JWT access tokens
- Refresh tokens
- Refresh token rotation
- User sessions
- Role model
- Permission model
- Role-permission mapping
- Basic account status management
- Authentication audit logs
- Authentication UI
- Security settings UI
- Session management UI

---

5. Explicit Non-Scope

Do **not** implement:

- Organizations
- Organization verification
- Organization dashboards
- Campaigns
- Needs
- Contributions
- Stripe
- Payments
- Volunteers
- Time Credits
- Rewards
- Full Admin Platform
- Enterprise SSO
- Passkeys
- Magic links
- Google/Microsoft/Apple login

Prepare extension points for future authentication methods, but do not implement them now.

---

6. Database Requirements

Extend the database with production-ready identity models.

Required models:

```
User
UserSession
RefreshToken
EmailVerificationToken
PasswordResetToken
Role
Permission
RolePermission
UserRole
AuthenticationAuditLog
ConsentRecord
```

Each model must include:

- UUID primary key
- createdAt
- updatedAt where applicable
- deletedAt where applicable
- indexes
- foreign keys
- constraints

---

7. User Model

User must support:

- id
- publicId
- firstName
- lastName
- email
- passwordHash
- phone
- country
- language
- timezone
- profileImageUrl
- status

- emailVerifiedAt
- lastLoginAt
- createdAt
- updatedAt
- deletedAt

Email must be unique.

Public ID must be unique.

Password hash must never be exposed.

---

8. User Statuses

Implement:

```
PENDING_EMAIL_VERIFICATION
ACTIVE
LOCKED
SUSPENDED
DELETED
```

Business rules:

- Pending users cannot access protected platform features.
- Locked users cannot login until unlocked.
- Suspended users cannot login.
- Deleted users cannot login.

---

9. Password Rules

Use Argon2id.

Password rules:

- Minimum 12 characters
- Allow passphrases
- Reject common weak passwords where possible
- Never log passwords
- Never return password hash
- Never store plain text passwords

---

10. Registration

Implement:

```
POST /v1/auth/register
```

Registration must:

- Validate email
- Validate password
- Reject duplicate email
- Hash password
- Create user
- Assign default Contributor role
- Create email verification token
- Queue verification email
- Write authentication audit log
- Return safe user response

Initial user status:

```
PENDING_EMAIL_VERIFICATION
```

---

11. Email Verification

Implement:

```
POST /v1/auth/verify-email
POST /v1/auth/resend-verification
```

Rules:

- Token is single-use
- Token expires
- User becomes ACTIVE after verification
- Audit log created
- Welcome email queued

---

12. Login

Implement:

```
POST /v1/auth/login
```

Login must:

- Validate credentials
- Reject unverified users
- Reject locked users
- Reject suspended users
- Create session
- Issue access token
- Issue refresh token
- Store hashed refresh token
- Update lastLoginAt
- Create authentication audit log

Invalid credentials must return a generic error.

Never reveal whether email exists.

---

13. Logout

Implement:

```
POST /v1/auth/logout
```

Logout must:

- Revoke current session
- Revoke current refresh token
- Create audit log

---

14. Refresh Tokens

Implement:

```
POST /v1/auth/refresh
```

Rules:

- Refresh token must be rotated
- Old refresh token must be revoked
- Reuse of revoked token should revoke the whole session
- Store only hashed refresh tokens
- Create audit log for refresh failures

---

15. Password Reset

Implement:

```
POST /v1/auth/forgot-password
POST /v1/auth/reset-password
```

Rules:

- Do not reveal whether email exists
- Token expires
- Token single-use
- New password must be hashed
- Existing sessions revoked after reset
- Password reset notification queued
- Audit log created

---

16. Current User

Implement:

```
GET /v1/users/me
PATCH /v1/users/me
```

User can update:

- firstName
- lastName
- phone
- country
- language
- timezone
- profileImageUrl

User cannot directly update:

- role
- permissions
- status
- emailVerifiedAt

---

17. Session Management

Implement:

```
GET /v1/users/me/sessions
DELETE /v1/users/me/sessions/{sessionId}
DELETE /v1/users/me/sessions
```

Users can:

- View active sessions
- Revoke one session
- Revoke all other sessions

Session data includes:

- device
- browser
- operating system
- ipAddress
- lastActivityAt
- createdAt

---

18. Roles

Create seed roles:

```
PLATFORM_OWNER
PLATFORM_ADMINISTRATOR
ORGANIZATION_OWNER
ORGANIZATION_ADMINISTRATOR
CAMPAIGN_MANAGER
VOLUNTEER_COORDINATOR
FINANCE_MANAGER
RECOGNITION_PARTNER
CONTRIBUTOR
VOLUNTEER
GUEST
```

Only assign `CONTRIBUTOR` by default on registration.

---

19. Permissions

Seed foundational permissions:

```
users.read_own
users.update_own
sessions.read_own
sessions.revoke_own
auth.login
auth.logout
auth.refresh
```

```
roles.read
permissions.read
audit.read_own
```

Do not seed full organization, campaign or payment permissions yet unless required as placeholders.

---

20. Authorization Infrastructure

Implement:

- JWT guard
- Role guard
- Permission guard
- Current user decorator
- Optional auth guard
- Public route decorator
- Permission decorator

Every protected endpoint must enforce authorization server-side.

---

21. Audit Logging

Create authentication audit records for:

- registration
- email verification
- login success
- login failure
- logout
- token refresh
- refresh token reuse
- password reset requested
- password reset completed
- session revoked
- account locked
- account suspended

Audit record includes:

- actorUserId if available
- targetUserId
- action
- result

- ipAddress
- userAgent
- requestId
- metadata
- createdAt

---

22. Notifications & Email

Use existing queue infrastructure.

Create email jobs for:

- email verification
- welcome email
- password reset
- password changed
- new login notification

Templates may be simple but must be real and functional.

No fake TODO template placeholders.

---

23. Frontend Pages

Create:

```
/register
/login
/logout
/verify-email
/resend-verification
/forgot-password
/reset-password
/account
/account/security
/account/sessions
```

UI requirements:

- Responsive
- Accessible
- Form validation
- Loading states
- Error states
- Success states
- Uses shared UI package

- Uses React Hook Form
- Uses Zod

---

24. Frontend Auth State

Implement:

- Auth provider
- Current user hook
- Login mutation
- Logout mutation
- Register mutation
- Refresh handling
- Protected route helper
- Public route helper

Do not store sensitive tokens insecurely.

---

25. API Client

Create typed frontend API client for auth and user endpoints.

Must handle:

- standard success response
- standard error response
- authentication errors
- token refresh
- logout on invalid session

---

26. Security Requirements

Implement:

- Argon2id password hashing
- Rate limiting on auth endpoints
- Generic login errors
- Secure token storage strategy
- Refresh token rotation
- Session revocation
- Input validation

- Audit logging
- No secrets in logs
- No password hashes in responses

---

27. Testing Requirements

Create tests for:

Backend Unit Tests

- password hashing
- token generation
- registration service
- login service
- refresh token rotation
- permission guard

Backend Integration Tests

- registration flow
- email verification flow
- login flow
- logout flow
- password reset flow
- session revocation flow

Frontend Tests

- registration form
- login form
- password reset form
- account page
- session page

Security Tests

- duplicate email rejected
- suspended user cannot login
- locked user cannot login
- revoked refresh token cannot be reused
- invalid password does not reveal user existence

---

28. OpenAPI

Document all endpoints with:

- request body
- response body
- status codes
- errors
- authentication requirements
- examples

---

29. Documentation

Update:

```
docs/authentication.md
docs/authorization.md
docs/session-management.md
docs/security.md
docs/api.md
docs/development-setup.md
```

Documentation must match implementation.

---

30. Acceptance Criteria

Package 2 is complete when:

- Users can register.
- Verification emails are sent.
- Users can verify email.
- Users can login.
- Users can logout.
- Refresh token rotation works.
- Users can reset password.
- Sessions are created and revoked.
- Role and permission infrastructure works.
- Protected endpoints reject unauthenticated users.
- Permission guard works.
- Authentication audit logs are created.
- Frontend authentication pages work.
- Tests pass.
- OpenAPI is updated.
- Documentation is accurate.

- No non-auth business features are implemented.

---

31. AI Execution Checklist

Before finishing, verify:

✓ No Organizations implemented.

✓ No Campaigns implemented.

✓ No Payments implemented.

✓ No Volunteers implemented.

✓ All auth endpoints validated.

✓ All protected endpoints require guards.

✓ Tokens are handled securely.

✓ Password hashes are never exposed.

✓ Audit logs are written.

✓ Tests pass.

✓ Docs updated.

---

32. Final Instruction to AI Coding Tool

Build the Authentication & Identity module only.

This package creates the security foundation for the entire Bidra platform.

Do not move into Organization, Campaign or Payment functionality.

The output must be production-ready enough that Development Package 3 can immediately build the Organization module on top of it without restructuring authentication.